

R Control Flow Statements

Building Blocks for Automated Decision-Making

Sherine Paul Arthur | 401147341 | paul_arthur.sherine@stud.hs-fresenius.de

2026-05-19

Where we left off

we got the data into R. and we saw how to describe it. so whats next?

a data frame sitting in your environment doesn't do anything on its own.

At some point you need your code to **make decisions** — check a condition, repeat a task, skip a row that doesn't belong.

That is what we are covering today.

What is control flow?

“Control flow refers to the order in which the instructions of a program are executed.”

— Lenovo Tech Glossary

By default R runs every line, top to bottom, no questions asked.
Control flow gives you three tools to change that:

- **Branching** — run code only when a condition is met
- **Looping** — repeat code automatically
- **Jumping** — skip or stop mid-loop

Branching — before we write any code

Imagine you have the final exam scores for everyone in this room.

- Score $\geq 90 \rightarrow$ excellent
- Score 60–89 $\rightarrow$ passed
- Score $< 60 \rightarrow$ needs follow-up

You would not check each one manually. You write the rule once and let R apply it to every row.

That rule is an `if / else` statement.

if / else

```
1 score <- 85
2
3 if (score >= 90) {
4   print("Excellent")
5 } else if (score >= 60) {
6   print("Passed")
7 } else {
8   print("Needs follow-up")
9 }
```

```
[1] "Passed"
```

R checks each condition top to bottom and stops at the first **TRUE**.

Inside `if()` always use `==` to compare, never `=` — `=` stores a value — `==` asks a question

ifelse() — the same logic, for an entire column

if / else works on **one value**. Your data has hundreds of rows.

```
1 # Final exam results
2 scores <- c(Ali=82, Luisa=47, Roopam=91, Mahdi=55, Cagan=68)
3
4 result <- ifelse(scores >= 60, "Pass", "Fail")
5 print(result)
```

```
Ali Luisa Roopam Mahdi Cagan
"Pass" "Fail" "Pass" "Fail" "Pass"
```

ifelse() applies the condition to every element at once.
Same logic as **if / else** — completely different scale.

Looping — the idea

we know how to get the mean of one column.

but what if you have 20 columns and need the mean of each?

```
1 data <- data.frame(  
2   sales   = c(200, 150, 300),  
3   expenses = c(100, 120, 90),  
4   profit  = c(100, 30, 210)  
5 )  
6  
7 for (col in names(data)) {  
8   cat("Mean of", col, ":", mean(data[[col]]), "\n")  
9 }
```

```
Mean of sales : 216.6667  
Mean of expenses : 103.3333  
Mean of profit : 113.3333
```

One loop. Works for 3 columns or 300.

for loops

Go through every item in a sequence and do something with each one.

```
1 # Weekly coffee budget per student (€)
2 coffee <- c(Sherine=18, Milad=22, Nwanneka=9, Mohammed=31)
3
4 for (name in names(coffee)) {
5   if (coffee[name] > 25) {
6     cat(name, "- needs an intervention\n")
7   } else {
8     cat(name, "- doing fine\n")
9   }
10 }
```

```
Sherine - doing fine
Milad - doing fine
Nwanneka - doing fine
Mohammed - needs an intervention
```

The loop variable takes each value one at a time. When it runs out — it stops.

while loops

Use when you don't know how many steps it will take.

```
1 balance <- 500
2 month  <- 0
3
4 while (balance < 1000) {
5   balance <- balance * 1.05 # 5% monthly return
6   month   <- month + 1
7 }
8
9 cat("Reached €1000 after", month, "months\n")
```

Reached €1000 after 15 months

Keeps going as long as the condition is **TRUE**. Stops the moment it becomes **FALSE**.

The infinite loop

If the condition never becomes **FALSE** — R freezes.

```
1 # DO NOT RUN THIS
2 x <- 1
3 while (x > 0) {
4   print(x)
5   # x never changes – this never stops
6 }
```

Always make sure something inside the loop **moves you toward the exit**.
In the savings example — **balance** keeps growing until the condition fails.

Jumping — interrupting a loop

Sometimes a loop needs to stop early or skip a step.

R gives you two tools for this: `break` and `next`.

break and next

break — exit the loop immediately:

```
1 submissions <- c(Saketh=4.2, Adineh=3.8, Shravanthi=-99, Bahadori=5.1)
2
3 for (name in names(submissions)) {
4   if (submissions[name] < 0) {
5     cat("Missing submission from", name, "- stopping.\n")
6     break
7   }
8   cat(name, "- submitted OK\n")
9 }
```

Saketh — submitted OK
Adineh — submitted OK
Missing submission from Shravanthi — stopping.

next — skip this one, keep going:

```
1 for (name in names(submissions)) {
2   if (submissions[name] < 0) next
3   cat(name, "- submitted OK\n")
4 }
```

Saketh — submitted OK
Adineh — submitted OK
Bahadori — submitted OK

Putting it all together

```
1 # -999 is a common placeholder for missing data
2 raw  <- c(320, -999, 415, 280, -999, 390, 610, 305)
3 clean <- c()
4
5 for (x in raw) {
6   if (x == -999) next # skip missing
7   if (x > 500) next  # skip outliers
8   clean <- c(clean, x)
9 }
10
11 cat("Original:", length(raw), "values\n")
```

Original: 8 values

```
1 cat("Clean:  ", length(clean), "values\n")
```

Clean: 5 values

```
1 cat("Mean:  ", round(mean(clean), 1), "\n")
```

Mean: 342

A **for** loop + **if** + **next** — three tools, one clean dataset.

Summary

- `if / else` — one decision on one value
- `else if` — more than two possible outcomes
- `ifelse()` — decision across a whole vector
- `for` — repeat over a known sequence
- `while` — repeat until a condition changes
- `break` — exit the loop immediately
- `next` — skip this iteration, keep going

Your turn — exercise

You have this vector:

```
1 numbers <- c(3, -1, 7, 0, -4, 9, 2, -8, 5, 1)
```

Write a `for` loop that prints `"positive"`, `"negative"`, or `"zero"` for each number.
Bonus: use `next` to skip any number greater than 6.

Solution

```
1 numbers <- c(3, -1, 7, 0, -4, 9, 2, -8, 5, 1)
2
3 for (n in numbers) {
4   if (n > 6) next
5
6   if (n > 0) {
7     cat(n, "→ positive\n")
8   } else if (n < 0) {
9     cat(n, "→ negative\n")
10  } else {
11    cat(n, "→ zero\n")
12  }
13 }
```

```
3 → positive
-1 → negative
0 → zero
-4 → negative
2 → positive
-8 → negative
5 → positive
1 → positive
```

References

- Grolemund, G. (2014). *Hands-on programming with R*. O'Reilly. <https://rstudio-education.github.io/hopr/>
- Hamelg. (2021). *Intro to R part 11: Control flow*. Kaggle. <https://www.kaggle.com/code/hamelg/intro-to-r-part-11-control-flow>
- Lenovo. (n.d.). *Control flow*. <https://www.lenovo.com/us/en/glossary/control-flow/>
- Peng, R. D. (2020). *R programming for data science*. Leanpub. <https://bookdown.org/rdpeng/rprogdatascience/>
- Phillips, N. D. (2018). *YaRrr! The Pirate's Guide to R*. <https://bookdown.org/ndphillips/YaRrr/>
- R Core Team. (2024). *R: A language and environment for statistical computing*. <https://www.R-project.org/>
- Wickham, H. (2019). *Advanced R* (2nd ed.). Chapman and Hall. <https://adv-r.hadley.nz/>

Access the materials

Scan to open the presentation and handout on your device:

Presentation:

https://sherinepaularthur.github.io/presents/control_flow_presentation.html

Handout:

https://sherinepaularthur.github.io/presents/control_flow_handout.html

Data Analysis for Decision-Making (SS 2026) | Prof. Dr. Stephan Huber

Speaker notes